Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: RICCARDO POLO Discussed with:

Report for Project 2

Due date: 23 March 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with OpenMP [20 points]

1.1. Exercise 1

In this exercise we are given a code that computes an approximation of π using numerical integration. The code uses a simple loop to compute the sum of terms that approximate the value of π. The goal of this exercise is to parallelize the code using OpenMP and to optimize it for better performance.

In the first part of the exercise it was asked to first parallelize the given code using a naive version that protects every loop update with `critical`. Here is the code snippet of the naive version that we implemented:

```
  #pragma omp parallel for
  for (uint64_t i = 0; i < N; ++i) {
    double x = (i + 0.5) * h;
    double term = 4.0 / (1.0 + x*x);

    #pragma omp critical
    pi += term;
  }
  pi *= h;

  double time = walltime() - time_start;

  int nt = omp_get_max_threads();
  printf("# pi N num_threads time [secs]\n");
  printf("%.15f %20"PRIu64" %3d %.8f\n", pi, N, nt, time);
```

The output of the naive version is as follows:

```
# pi N num_threads time [secs]
3.141592653590757 1000000000 64 640.70646905
```

We can see that the naive version is very slow. This is because every update to the shared variable `pi` is protected by a critical section, which causes a lot of contention and serialization among the threads. Each thread has to wait for the others to finish updating `pi`, which leads to a significant slowdown in performance.

In the second part in was asked to create an optimized version that accumulates locally and enters critical only once per thread. Here is the code snippet of the optimized version that we implemented:

```
  #pragma omp parallel
  {
    double sum = 0.0;

    #pragma omp for
    for (uint64_t i = 0; i < N; ++i) {
      double x = (i + 0.5) * h;
      sum += 4.0 / (1.0 + x*x);
    }

    #pragma omp critical
    pi += sum; // once per thread
  }
  pi *= h;

  double time = walltime() - time_start;

  int nt = omp_get_max_threads();
  printf("# pi N num_threads time [secs]\n");
  printf("%.15f %20"PRIu64" %3d %.8f\n", pi, N, nt, time);
```

In this version we have a local variable `sum` that is private to each thread, and we only enter the critical section once per thread to update the shared variable `pi`. This reduces the contention on the critical section and allows for better performance. The output of the optimized version is as follows:

```
# pi N num_threads time [secs]
3.141592653589771 1000000000 64 0.19858712
```

We can see that the optimized version is significantly faster than the naive version. The optimized version allows for better utilization of the available threads and reduces the overhead caused by the critical section.

We are then asked to implement a version that uses the reduction clause. Here is the code snippet of the version that uses the reduction clause:

```
  #pragma omp parallel for reduction(+:pi)
#pragma omp parallel for reduction(+:sum)
  for (uint64_t i = 0; i < N; ++i) {
    double x = (i + 0.5)*h;
    sum += 4.0 / (1.0 + x*x);
  }
  pi = sum*h;
  double time = walltime() - time_start;

  int nt = omp_get_max_threads();
  printf("# pi N num_threads time [secs]\n");
  printf("%.15f %20"PRIu64" %3d %.8f\n", pi, N, nt, time);
```

In this version we use the `reduction` clause to automatically handle the accumulation of the variable `pi` across threads. The reduction clause takes care of creating a private copy of the variable for each thread and then combining the results at the end of the parallel region. This allows for a more concise and efficient implementation. The output of the version that uses the reduction clause is as follows:

```
# pi N num_threads time [secs]
3.141592653589771 1000000000 64 0.11437800
```

We can see that the optimized version and the version that uses the reduction clause are significantly faster than the naive version. The reduction clause version is also more concise and easier to read, making it a good choice for this type of problem.
In table 1 we summarize the results of the different versions of the code.

Version	pi	num_threads	time [secs]
Serial	3.141592653589971	1	1.88825762
Naive	3.141592653590757	64	640.706
Optimized	3.141592653589771	64	0.19858712
Reduction	3.141592653589771	64	0.11437800

Table 1: Results of the different versions of the code for computing π

1.2. Exercise 2

In the second part of the exercise we are asked to perform a weak and strong scaling study.

1.2.1. Strong scaling study

In this context a strong scaling study is performed by keeping the problem size fixed and increasing the number of threads. The goal is to see how the performance of the code improves as we increase the number of threads. In our case, we can keep the value of N fixed and measure the execution time for different numbers of threads. We can then plot the speedup and efficiency as a function of the number of threads to analyze the strong scaling behavior of our code.
Here are outputs.
Strong naive

```
  # pi N num_threads time [secs]
3.141592653589634 1000000000 1 6.93277019
3.141592653589634 1000000000 2 76.46882527
3.141592653589634 1000000000 4 197.49973289
3.141592653589634 1000000000 8 283.80093214
3.141592653589634 1000000000 16 403.18691688
3.141592653589634 1000000000 32 501.40493621
3.141592653589634 1000000000 64 678.71737772
```

Strong optimized)

```
# pi N num_threads time [secs]
3.141592653589771 1000000000 1 2.26213344
3.141592653589771 1000000000 2 1.26299218
3.141592653589771 1000000000 4 0.67451669
3.141592653589771 1000000000 8 0.33567476
3.141592653589771 1000000000 16 0.17712136
3.141592653589771 1000000000 32 0.10922673
3.141592653589771 1000000000 64 0.10740197
```

Strong reduction)

```
  # pi N num_threads time [secs]
3.141592653589772 1000000000 1 2.19981237
3.141592653589772 1000000000 2 1.26300334
3.141592653589772 1000000000 4 0.67434969
3.141592653589772 1000000000 8 0.33552001
3.141592653589772 1000000000 16 0.17785895
3.141592653589772 1000000000 32 0.12054630
3.141592653589772 1000000000 64 0.07776448
```

Weak naive)

```
  3.141592653589632 100000 1 0.00118749
3.141592653589632 200000 2 0.02662989
3.141592653589632 400000 4 0.13208618
3.141592653589632 800000 8 0.22754601
3.141592653589632 1600000 16 0.62820553
3.141592653589632 3200000 32 2.44359762
3.141592653589632 6400000 64 5.08367890
```

Weak optimized)

```
  # pi N num_threads time [secs]
3.141592653589791 100000 1 0.00026265
3.141592653589791 200000 2 0.00038406
3.141592653589791 400000 4 0.00040525
3.141592653589791 800000 8 0.00053178
3.141592653589791 1600000 16 0.00072227
3.141592653589791 3200000 32 0.01441353
3.141592653589791 6400000 64 0.02746264
```

Weak reduction)

```
  # pi N num_threads time [secs]
3.141592653589791 100000 1 0.00025960
3.141592653589791 200000 2 0.00037925
3.141592653589791 400000 4 0.00044645
3.141592653589791 800000 8 0.00058488
3.141592653589791 1600000 16 0.00804680
3.141592653589791 3200000 32 0.01407022
3.141592653589791 6400000 64 0.02701498
```

2. The Mandelbrot set using OpenMP [20 points]

2.1. Exercise 1 and 2

Here we are asked to complete the serial version of the Mandelbrot set code in the missing code part and to count the total number of iterations in order to correctly compute the benchmark statistics.
In the following code snippet we show the completed version of the serial code for computing the Mandelbrot set:

```
double time_start = walltime();
  // do the calculation
  cy = MIN_Y;
  for (j = 0; j < IMAGE_HEIGHT; j++) {
    cx = MIN_X;
    for (i = 0; i < IMAGE_WIDTH; i++) {
      x = cx;
      y = cy;
      x2 = x * x;
      y2 = y * y;
      // compute the orbit z, f(z), f^2(z), f^3(z), ...
      // count the iterations until the orbit leaves the circle |z|=2.
      // stop if the number of iterations exceeds the bound MAX_ITERS.
      int n = 0;
      // TODO
      // >>>>>>>> CODE IS MISSING
       while (x2 + y2 <= 4.0 && n < MAX_ITERS) {
         y = 2.0 * x * y + cy;
         x = x2 - y2 + cx;
         x2 = x * x;
         y2 = y * y;
         n++;
       }
       nTotalIterationsCount += n;
      // <<<<<<<< CODE IS MISSING
      // n indicates if the point belongs to the mandelbrot set
      // plot the number of iterations at point (i, j)
      int c = ((long)n * 255) / MAX_ITERS;
      ppm_plot(&pPpm, i, j, c, c, c);
      cx += fDeltaX;
    }
    cy += fDeltaY;
  }
  double time_end = walltime();
```

In the while loop, we are iterating through the orbit of the point c under the function $f(z) = z^2 + c$. The variables x and y represent the real and imaginary parts of the complex number z, respectively. The equations used in the while loop are derived from the definition of complex multiplication and addition. Specifically, the function $f(z)$ can be expressed in terms of its real and imaginary components as follows:ù

$$\begin{aligned} f(z) &= z^2 + c \\ &= (x + yi)^2 + (cx + cyi) \\ &= (x^2 - y^2 + cx) + (2xy + cy)i \end{aligned}$$

But why does this hold also for $f_z(c)^3, ...$? This is because the function $f(z)$ is defined recursively, and each iteration of the function applies the same transformation to the complex number z. Therefore, when we compute $f^2(z)$, we are applying the function f to the result of $f(z)$, which means that we are applying the same equations for the real and imaginary parts at each iteration. This recursive nature of the function allows us to use the same equations for each iteration in the while loop, regardless of how many times we apply the function. From this, we can see that the real part of f(z) is given by $x^2 - y^2 + cx$, and the imaginary part is given by $2xy + cy$. Therefore, in the while loop, we update the values of x and y according to these equations to compute the next point in the orbit. The loop continues until the magnitude of z (which is

$\sqrt{x^2 + y^2}$) exceeds 2, indicating that the orbit is unbounded, or until we reach the maximum number of iterations, which indicates that the point c is likely to belong to the Mandelbrot set.
Finally, we are asked to count the total number of iterations in order to correctly compute the benchmark statistics. This is done by introducing a variable nTotalIterationsCount that accumulates the number of iterations for each point in the complex plane. This variable is incremented by n, which represents the number of iterations for the current point (i, j). By keeping track of the total number of iterations, we can compute the benchmark statistics more accurately, as it gives us a measure of the computational effort required to compute the Mandelbrot set for the given parameters.

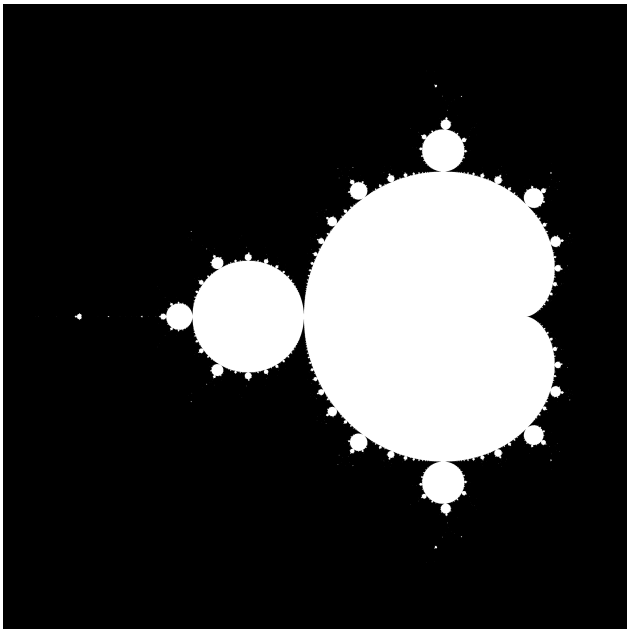


Figure 1: Serial Mandelbrot Set

2.2. Exercise 3

In the third part of the exercise we are asked to parallelize the Mandelbrot set code using `OpenMP`. The goal is to improve the performance of the code by utilizing multiple threads to compute the Mandelbrot set in parallel. Here is the code snippet of the parallelized version of the Mandelbrot set code using `OpenMP`:

```
int main(int argc, char **argv) {
ppm_data_t pPpm;
ppm_create(&pPpm, IMAGE_WIDTH, IMAGE_HEIGHT);

double fDeltaX = (MAX_X - MIN_X) / (double)IMAGE_WIDTH;
double fDeltaY = (MAX_Y - MIN_Y) / (double)IMAGE_HEIGHT;

long nTotalIterationsCount = 0;

double time_start = walltime();
// do the calculation
#pragma omp parallel for reduction(+:nTotalIterationsCount) schedule(dynamic)
for (long j = 0; j < IMAGE_HEIGHT; j++) {
  double cy = MIN_Y + j * fDeltaY;

  for (long i = 0; i < IMAGE_WIDTH; i++) {
    double cx = MIN_X + i * fDeltaX;

    double x = cx;
    double y = cy;
    double x2 = x * x;
```

```
    double y2 = y * y;

    int n = 0;
    while (x2 + y2 <= 4.0 && n < MAX_ITERS) {
      y = 2.0 * x * y + cy;
      x = x2 - y2 + cx;
      x2 = x * x;
      y2 = y * y;
      n++;
    }

    nTotalIterationsCount += n;

    int c = ((long)n * 255) / MAX_ITERS;
    ppm_plot(&pPpm, i, j, c, c, c);
  }
}
double time_end = walltime();
```

Metric	Serial Version	Parallel Version
Total time [seconds]	330.726	6.70717
Total number of iterations	113610974266	113638786651
Average time per pixel [seconds]	1.97128e-05	3.99779e-07
Average time per iteration [seconds]	2.91104e-09	5.90219e-11
Iterations per second	3.4352e+08	1.69429e+10
MFlop/s	2748.16	135543

Table 2: Comparison of the serial and parallel versions of the Mandelbrot set code

The results (see table 2) show a significant performance improvement achieved by parallelizing the code using `OpenMP`. The total time taken to compute the Mandelbrot set is reduced from 330.726 seconds in the serial version to 6.70717 seconds in the parallel version, which is a substantial decrease in execution time. The total number of iterations is slightly higher in the parallel version, which could be due to differences in the order of execution and how the iterations are distributed among the threads. The average time per pixel and average time per iteration are also significantly reduced in the parallel version, indicating that the computation is much faster when utilizing multiple threads. The iterations per second and MFlop/s metrics show a dramatic increase in performance, with the parallel version achieving 1.69429e+10 iterations per second and 135543 MFlop/s compared to 3.4352e+08 iterations per second and 2748.16 MFlop/s in the serial version. This performance improvement can be attributed to the ability of `OpenMP` to effectively utilize the available CPU cores, allowing for concurrent execution of the computations required to determine whether each point belongs to the Mandelbrot set. By distributing the workload across multiple threads, the parallel version can significantly reduce the time required to compute the Mandelbrot set, especially for larger image sizes and higher iteration counts.

2.2.1. Strong scaling analysis

3. Parallel histogram calculation using `OpenMP` [15 points]

4. Parallel loop dependencies with `OpenMP` [15 points]

5. Quicksort using `OpenMP` tasks [20 points]